

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Computación gráfica, visión computacional y multimedia				
TÍTULO DE LA PRÁCTICA:	<i>Transformaciones Geométricas 2D</i>				
NÚMERO DE PRÁCTICA:	02	AÑO LECTIVO:	2026A	NRO. SEMESTRE:	IX
FECHA DE PRESENTACIÓN	03/05/2026	HORA DE PRESENTACIÓN			
INTEGRANTE (s): Maxs Sebastian Joaquin Forocca Mamani				NOTA:	
DOCENTE(s): <i>RENE ALONSO NIETO VALENCIA</i>					

SOLUCIÓN Y RESULTADOS
I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS 1. Descripción de la Aplicación La aplicación es un juego de plataformas 2D con mecánicas de puzzle basadas en transformaciones geométricas. El jugador controla un personaje que puede moverse, saltar, cambiar de tamaño y debe evitar trampas mientras recolecta monedas para alcanzar la meta de la escena. Características principales: • Movimiento lateral y salto del jugador (traslación 2D). • Cambio de tamaño interactivo con teclas W / S (escalado 2D). • Trampas estáticas con rebote y trampa rodante con rotación física. • Efecto Parallax en las capas de fondo del escenario. • Sistema de vida (barra de salud), monedas, pausa y pantalla de victoria. • Cámara con seguimiento suavizado y límites de mapa. 2. Estructura del Código y Transformaciones Geométricas a. Traslación 2D — Movimiento del Jugador

El movimiento del personaje es la implementación directa de la traslación bidimensional: el objeto jugador se desplaza frame a frame modificando su posición en el plano XY.

PlayerController.cs

Archivo	PlayerController.cs
Clase	PlayerController : MonoBehaviour
Responsabilidad	Controlador principal del jugador. Lee input, aplica traslación horizontal y vertical (salto) y coordina el escalado.
Transformación	Traslación (Move, Jump) + Escalado (SizeControl) + Reflejo horizontal (Flip)
Métodos clave	Move(), Jump(), SizeControl(), Flip(), Animate()

Fragmento clave — Traslación horizontal (Move):

```
void Move() {  
    float targetSpeed = moveInput * stats.moveSpeed;  
    if(!isGrounded) targetSpeed *= airControlMultiplier;  
    rb.velocity = new Vector2(targetSpeed, rb.velocity.y);  
    // x' = moveInput * speed → traslación en eje X  
}
```

Fragmento clave — Traslación vertical (salto):

```
void Jump() {  
    isGrounded = Physics2D.OverlapCircle(...);  
    if (Input.GetButtonDown("Jump") && isGrounded)  
        rb.velocity = new Vector2(rb.velocity.x, stats.jumpForce);  
    // y' = jumpForce → traslación en eje Y  
}
```

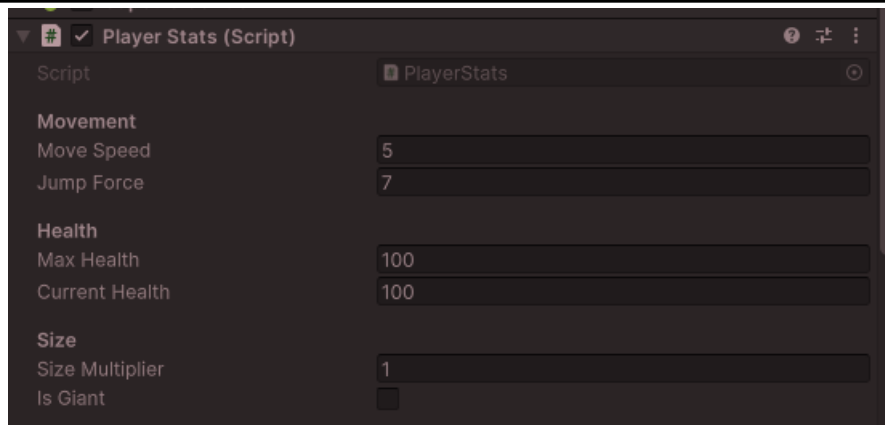


PlayerStats.cs

Archivo	PlayerStats.cs
Clase	PlayerStats : MonoBehaviour
Responsabilidad	Almacena las estadísticas del jugador (velocidad, fuerza de salto, vida, tamaño) y expone el método ChangeSize para el escalado.
Transformación	Escalado 2D (ChangeSize) — transforma localScale del objeto
Métodos clave	ChangeSize(float multiplier), TakeDamage(int), Die(), InitializeHealth()

Fragmento clave — Escalado uniforme (ChangeSize):

```
public void ChangeSize(float multiplier) {
    sizeMultiplier = multiplier;
    transform.localScale = Vector3.one * sizeMultiplier;
    // Matriz de escala: p = q = sizeMultiplier
    // x' = p·x, y' = q·y (escala uniforme)
    float growthFactor = 1f + (sizeMultiplier - 1f) * 0.25f;
    moveSpeed = growthFactor * 5f;
    jumpForce = growthFactor * 7f;
}
```

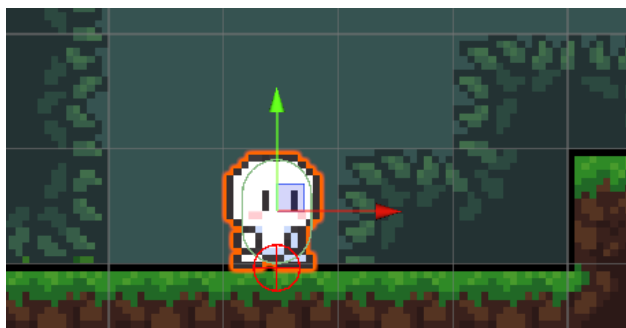


b. Escalado 2D — Control de Tamaño del Jugador

La mecánica de cambio de tamaño es la aplicación directa del escalado afín. El jugador puede presionar W para duplicar su tamaño ($p = q = 2$) o S para restaurarlo ($p = q = 1$). Esta transformación también ajusta las estadísticas de movimiento proporcionalmente.

El método SizeControl en PlayerController.cs actúa como activador de la transformación de escala, delegando el cálculo a PlayerStats.cs:

```
void SizeControl() {
    if (Input.GetKeyDown(KeyCode.W) && !stats.isGiant) {
        stats.ChangeSize(2f); // escala × 2
        stats.isGiant = true;
    }
    if (Input.GetKeyDown(KeyCode.S) && stats.isGiant) {
        stats.ChangeSize(1f); // escala × 1 (restaurar)
        stats.isGiant = false;
    }
}
```



c. Rotación 2D — Obstáculos Dinámicos

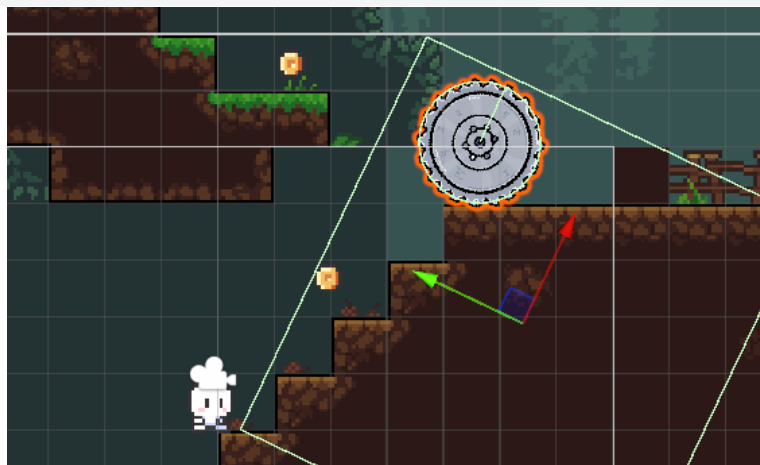
La rotación se implementa en los obstáculos del escenario. Unity delega el cálculo de la matriz de rotación al motor de física (Rigidbody2D), pero el concepto subyacente es la transformación rotacional del marco teórico.

RollingTrap.cs

Archivo	RollingTrap.cs
Clase	RollingTrap : MonoBehaviour
Responsabilidad	Trampa rodante activada por el jugador. Al detectar al jugador en su trigger, aplica una fuerza de traslación (AddForce) y un torque de rotación (AddTorque).
Transformación	Rotación 2D (AddTorque) + Traslación (AddForce)
Métodos clave	OnTriggerEnter2D(), OnCollisionEnter2D()

Fragmento clave — Rotación con AddTorque:

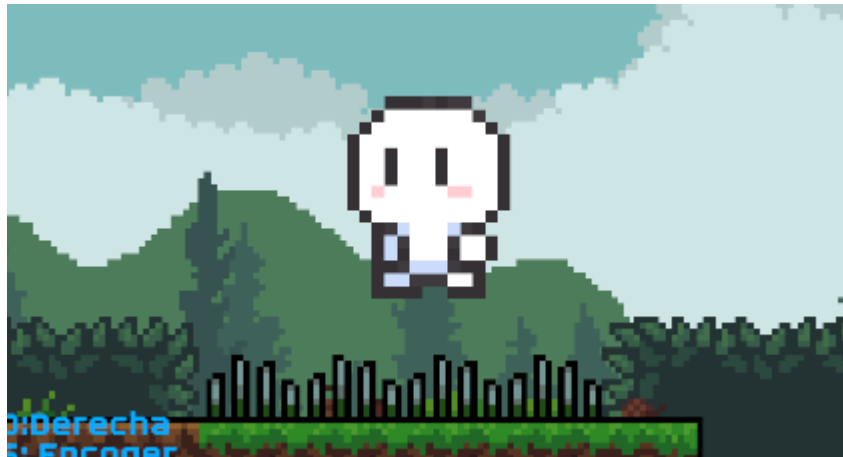
```
void OnTriggerEnter2D(Collider2D other) {
    if (other.CompareTag("Player") && !isRolling) {
        isRolling = true;
        rb.AddForce(Vector2.left * force, ForceMode2D.Impulse);
        // Traslación: empuje hacia la izquierda
        rb.AddTorque(torque, ForceMode2D.Impulse);
        // Rotación:  $\theta$  aumenta cada frame por el torque
    }
}
```



Trap.cs

Archivo	Trap.cs
---------	---------

Clase	Trap : MonoBehaviour
Responsabilidad	Trampa estática. Detecta colisión con el jugador, aplica daño y genera un rebote (traslación vertical instantánea).
Transformación	Traslación (rebote vertical en eje Y)
Métodos clave	OnCollisionEnter2D()



d. Efecto Parallax — Traslación Diferencial de Capas

El efecto Parallax es una técnica que aplica traslaciones de distinta velocidad a múltiples capas de fondo para simular profundidad. Es directamente el concepto de traslación bidimensional con factor de escala aplicado al desplazamiento.

ParallaxEffect.cs

Archivo	ParallaxEffect.cs
Clase	ParallaxEffect : MonoBehaviour
Responsabilidad	Aplica traslación diferencial a cada capa de fondo según el movimiento de la cámara. Incluye lógica de tiling infinito para evitar bordes visibles.
Transformación	Traslación 2D diferencial (Parallax) — transform.Translate()
Métodos clave	LateUpdate(), Start()

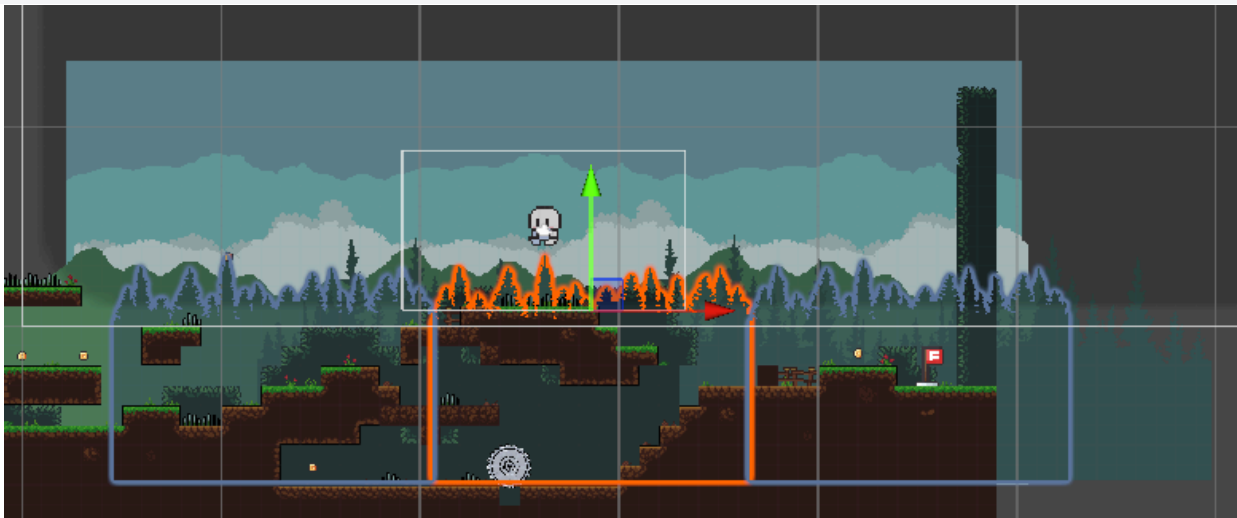
Fragmento clave — Traslación diferencial:

```
void LateUpdate() {
    float deltaX = (camaraTransform.position.x -
                    previousCamaraPosition.x) * parallaxMultiplier;
    // Δx capa = Δx cámara × α → traslación proporcional
```

```

    transform.Translate(new Vector3(deltaX, 0, 0));
    previousCamaraPosition = camaraTransform.position;
    // Lógica de tiling para repetición infinita del fondo
    float moveAmount = camaraTransform.position.x * (1 - parallaxMultiplier);
    if(moveAmount > startPosition + spriteWidth)
        transform.Translate(new Vector3(spriteWidth, 0, 0));
    else if(moveAmount < startPosition - spriteWidth)
        transform.Translate(new Vector3(-spriteWidth, 0, 0));
}

```



e. Seguimiento de Cámara — Traslación Suavizada

CameraFollow.cs

Archivo	CameraFollow.cs
Clase	CameraFollow : MonoBehaviour
Responsabilidad	La cámara sigue al jugador mediante interpolación suavizada (SmoothDamp) y se mantiene dentro de límites definidos (Clamping). Es una traslación continua de la cámara hacia la posición objetivo.
Transformación	Traslación 2D suavizada de la cámara (SmoothDamp + Clamp)
Métodos clave	LateUpdate(), OnDrawGizmosSelected()

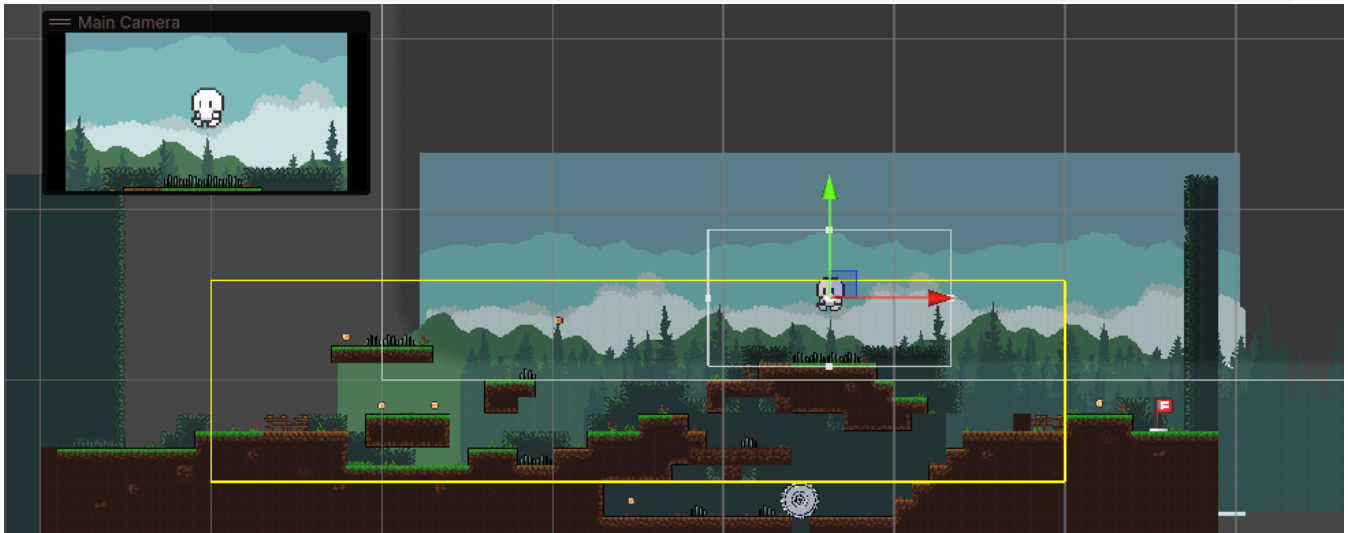
Fragmento clave — Traslación suavizada:

```

void LateUpdate() {
    Vector3 targetPosition = new Vector3(
        player.position.x, player.position.y, transform.position.z);
    targetPosition.x = Mathf.Clamp(targetPosition.x, minX, maxX);
}

```

```
targetPosition.y = Mathf.Clamp(targetPosition.y, minY, maxY);
// Traslación suavizada: posición interpolada frame a frame
transform.position = Vector3.SmoothDamp(
    transform.position, targetPosition, ref velocity, smoothTime);
}
```



f. Sistemas de Soporte y Gestión del Juego

Los siguientes scripts no implementan transformaciones geométricas directamente, pero son indispensables para el funcionamiento de la aplicación y la experiencia del jugador:

GameManager.cs + GameState.cs

Archivo	GameManager.cs / GameState.cs
Clase	GameManager : MonoBehaviour (Singleton) / enum GameState
Responsabilidad	Controla el estado global del juego (Playing, Paused, Win). Gestiona Time.timeScale para congelar o reanudar el tiempo. Es el árbitro central que habilita o deshabilita las transformaciones.
Transformación	Sin transformación directa — habilita/deshabilita la ejecución de transformaciones vía IsPlaying()
Métodos clave	PauseGame(), ResumeGame(), WinGame(), RestartLevel(), IsPlaying()

UIManager.cs

Archivo	UIManager.cs
---------	--------------

Clase	UIManager : MonoBehaviour (Singleton)
Responsabilidad	Administra todos los elementos de la interfaz gráfica: barra de vida, contador de monedas, panel de pausa y panel de victoria.
Transformación	Sin transformación geométrica directa
Métodos clave	UpdateHealth(), AddCoin(), ShowPausePanel(), ShowWinPanel()

Goal.cs + Coin.cs

Archivo	Goal.cs / Coin.cs
Clase	Goal : MonoBehaviour / Coin : MonoBehaviour
Responsabilidad	Goal detecta cuando el jugador llega a la meta (trigger) y activa la victoria. Coin detecta la colisión del jugador, suma puntos y se destruye.
Transformación	Sin transformación directa — objetos del puzzle que reaccionan a la traslación del jugador
Métodos clave	OnTriggerEnter2D()

PauseMenu.cs + CursorManager.cs

Archivo	PauseMenu.cs / CursorManager.cs
Clase	PauseMenu : MonoBehaviour / CursorManager : MonoBehaviour (Singleton)
Responsabilidad	PauseMenu escucha la tecla Escape y delega en GameManager. CursorManager oculta/muestra el cursor según el estado del juego.
Transformación	Sin transformación geométrica directa
Métodos clave	Update() / HideCursor(), ShowCursor()

CounterCoins.cs

Archivo	CounterCoins.cs
Clase	CounterCoins : MonoBehaviour
Responsabilidad	Componente auxiliar de UI que sincroniza el texto de monedas entre el HUD y otro elemento de la escena.

Transformación	Sin transformación geométrica directa
Métodos clave	Update ()
3. Enlace a github: https://github.com/MaxsForocca/LabComputacionGrafica 4. Video Drive:  nie-bdde-jzq (2026-05-03 18:19 GMT-5)	

II. SOLUCIÓN DEL CUESTIONARIO

III. CONCLUSIONES

El desarrollo de este laboratorio sobre transformaciones geométricas 2D permitió integrar de manera efectiva conceptos fundamentales de álgebra lineal y cinemática dentro de un entorno de videojuegos funcional. A través de la implementación precisa de traslaciones —como el movimiento del jugador, el seguimiento suavizado de la cámara y el efecto parallax—, escalados dinámicos para alterar las propiedades del personaje y rotaciones físicas para obstáculos, se logró materializar una jugabilidad coherente donde la manipulación espacial es la protagonista. Este ejercicio no solo validó la aplicación técnica de matrices y vectores en la lógica de programación, sino que también demostró cómo la correcta gestión de estas transformaciones geométricas resulta esencial para construir una experiencia interactiva fluida, dinámica y visualmente rica en un ecosistema bidimensional.

TROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA



UNIVERSIDAD NACIONAL DE SAN AGUSTIN
FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA



Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación

Aprobación: 2022/03/01

Código: GUIA-PRLE-001

Página: 11